430.211 Programming Methodology (프로그래밍 방법론)

Sorting Algorithm part 2

Lab2 #03

Fall 2025

Machine Intelligence & Data science LAB

Outline

- **Recursion**
  - **What is recursion?**
- Incremental Algorithm
  - Merge sort
  - Quick sort
- Linear time sorting algorithm
  - Counting, bucket, radix sorting…

Recursion

- Recursion is the process where a **function calls itself**.

- It is often used to solve problems by **breaking them into smaller subproblems**.

Recursion: Sum function

- Recursion can be defined by two key components:
  - **Base case**
    - The condition that stops the recursion.
  - **Recursive step**
    - Calling the function on a smaller version of itself
    - (ex) sum(n) = n + sum(n-1)

Base case →

Recursive step →

$$\begin{cases} a_1 = 1 \\ a_n = a_{n-1} + 1 \quad (n \in \{2, 3, 4, \ldots\}) \end{cases}$$

Recursion : Factorial function

- For integer $n \geq 1$, the factorial function is defined by

$$n! = (n) \cdot (n-1) \cdot (n-2) \cdots (1).$$

- It can be represented **recursively**.

Base case → $1! = 1$

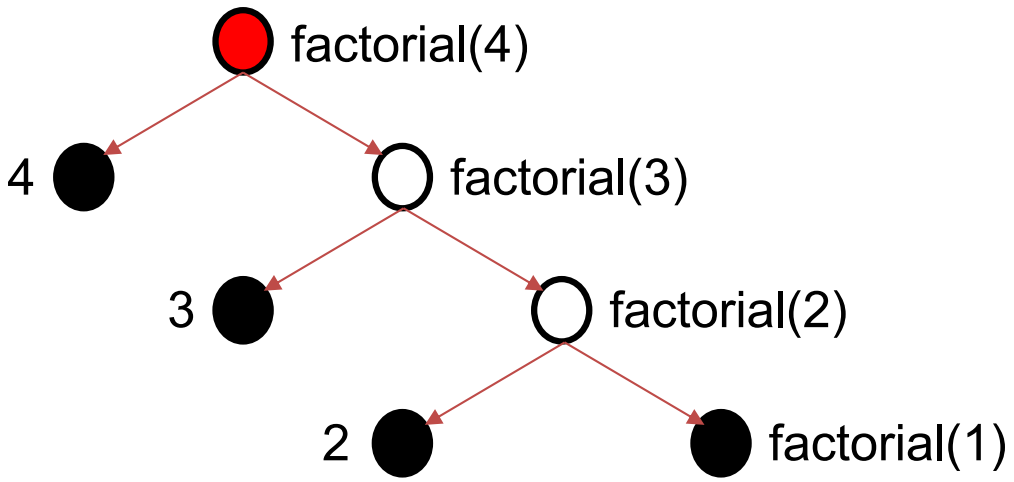
Recursive step → $n! = n \cdot (n-1)!$

Recursion : Factorial

- Pseudocode

```
factorial(n)
    if n == 1
        return 1
    else
        return = n * factorial(n − 1)
```

- Tree structure



Recursion : Factorial

- Code

Not recursion

Base case

Recursive step

```cpp
#include <iostream>
#include <string>
using namespace std;
int factorial_1(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
        result *= i;
    return result;
}
int factorial_2(int n)
{
    if (n == 1)
        return 1;
    else
        return n*factorial_2(n-1);
}
int main()
{
    int n;
    cout << "Enter a positive integer :" << endl;
    cin >> n;
    cout << "For loop result : " << factorial_1(n) << endl;
    cout << "Recursive result : " << factorial_2(n) << endl;
    return 0;
```

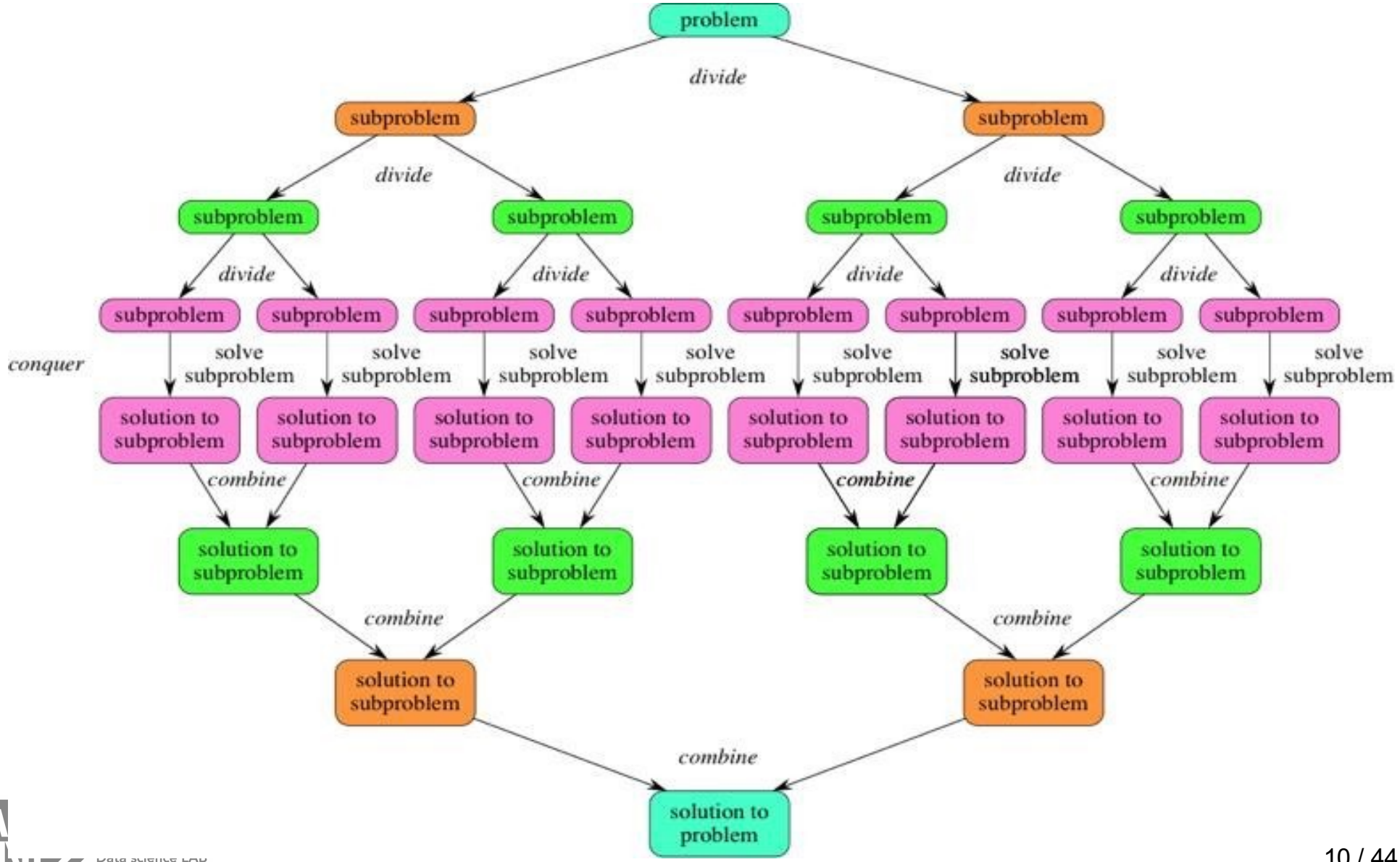
Outline

- Recursion
  - What is recursion?
- **Incremental Algorithm (Divide and Conquer algorithms)**
  - Merge sort
  - Quick sort
- Linear time sorting algorithm
  - Counting, bucket, radix sorting…

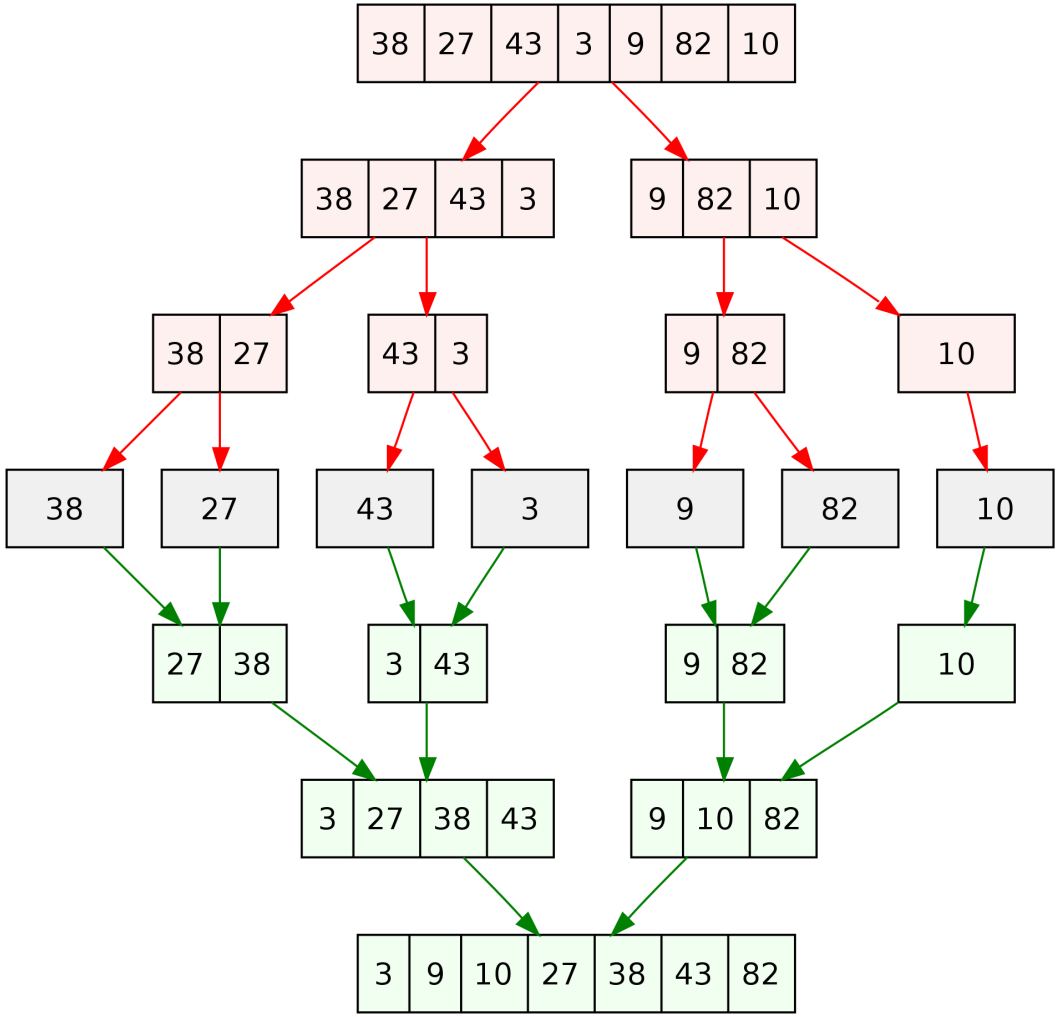
Divide and Conquer

- **Divide**
  - Break the problem into smaller sub-problems
- **Conquer**
  - Solve each sub-problem by recursively calling the function until it is solved
- **Combine**
  - Merge the solutions of the sub-problems to get the final solution for the whole problem.

Divide and Conquer

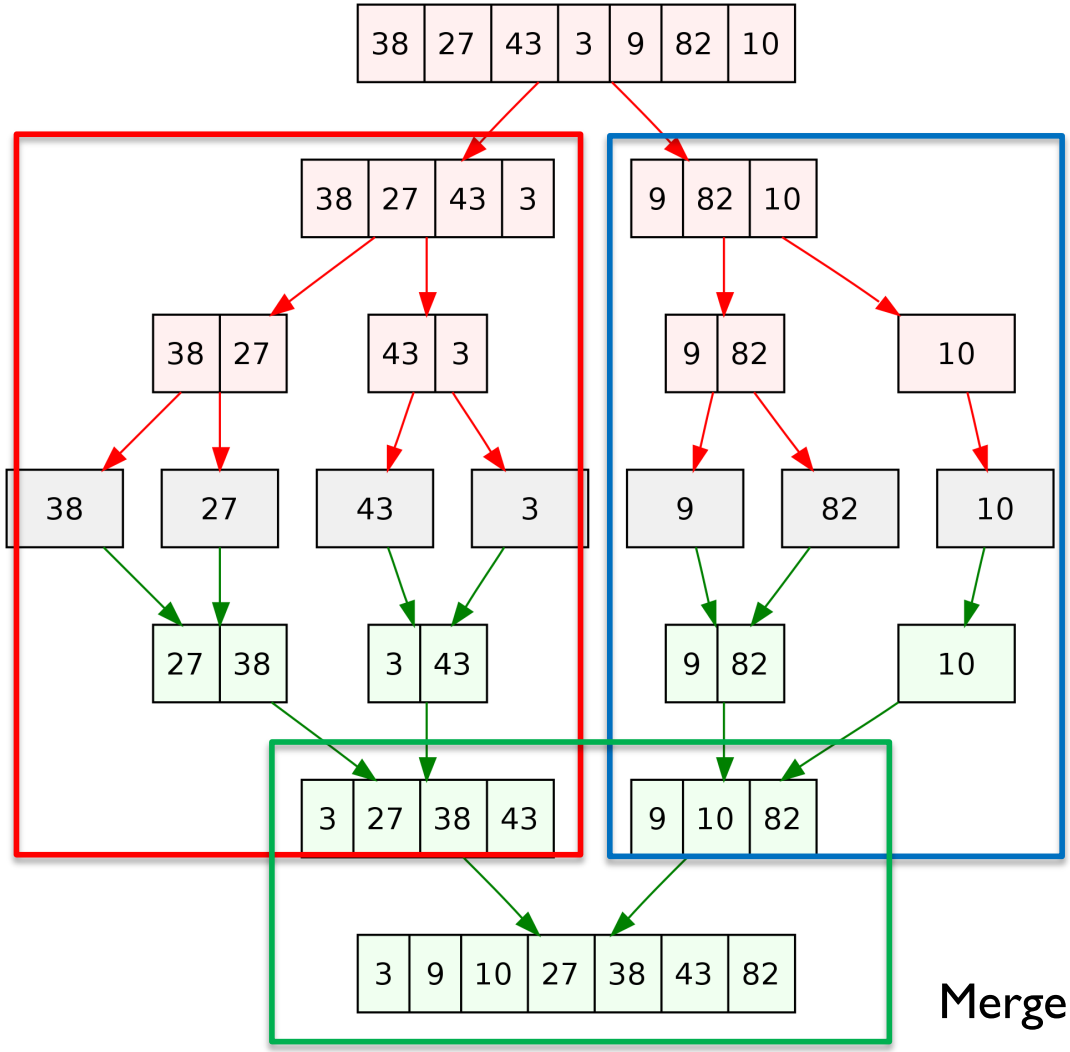


Merge Sort



- Divide the array into halves until each sub-array has length 1
- Then, keep merging the halves together in the correct order

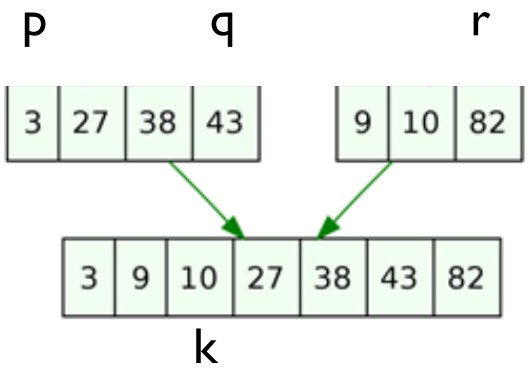
Merge Sort




```
void merge_sort(int* array, int l, int r) {
        if (l < r)
        {
                int q = (l + r) / 2;
                merge_sort(array, l, q);
                merge_sort(array, q + 1, r);
                merge(array, l, q, r);
        }
}
```

Merge two sorted list

Merge Sort




```
MERGE(A, p, q, r)                                   p: left, q: pivot, r: right index
1   n1 ← q − p + 1
2   n2 ← r − q
3   create arrays L[ 0, ..., n1 ] and R[ 0, ..., n2 ]
4   for i ← 0 to n1 − 1
5       do L[i] ← A [p + i]
6   for j ← 0 to n2 − 1                             Fill L and R
7       do R[j] ← A [q + j +1]                      (Copy array values into L,R)
8   i ← 0
9   j ← 0
10  for k ← p to r
11      do if L[i] ≤ R[j]
12            then A[k] ← L[i]                      Sorting
13                 i ← i + 1
14            else A[k] ← R[j]
15                 j ← j + 1
```

Merge Sort

left pivot right

```
void merge(int* array, int p, int q, int r) {
    int n1 = q - p + 1;
    int n2 = r - q;

    ////////// IMPLEMENT HERE //////////

    ////////////////////////////////////
}
```

```
void merge_sort(int* array, int l, int r) {
        if (l < r)
        {
                int q = (l + r) / 2;
                merge_sort(array, l, q);
                merge_sort(array, q + 1, r);
                merge(array, l, q, r);
        }
}
```

Merge Sort

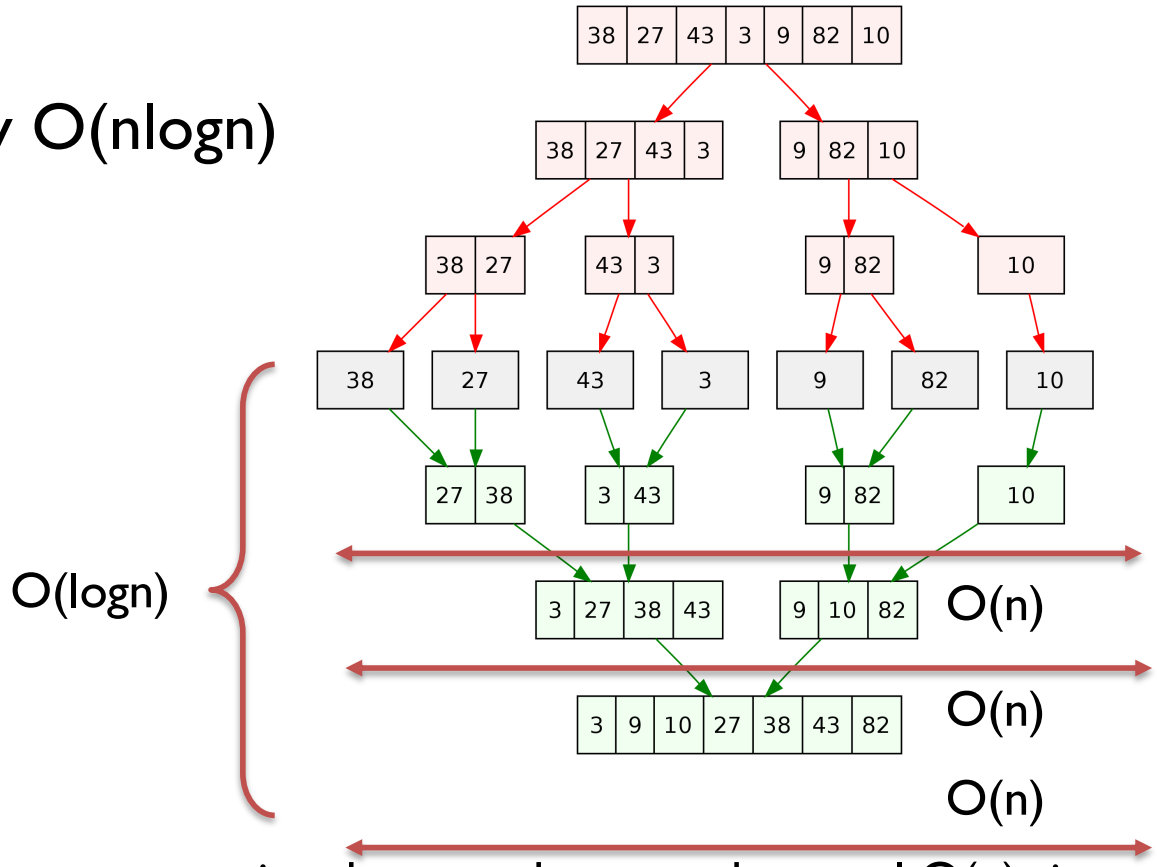
left pivot right

```
void merge(int* array, int p, int q, int r) {
    int n1 = q - p + 1;
    int n2 = r - q;

    ////////// IMPLEMENT HERE //////////
    int L[n1];
    int R[n2];
    for (int i = 0; i < n1; i++){
        L[i]= array[p+i];
    }
    for (int i = 0; i < n2; i++){
        R[i]= array[q+i+1];
    }
    int i = 0;
    int j = 0;

    for (int k = p; k <= r; k++){
        if (i == n1) array[k] = R[j++];
        else if (j == n2 || L[i] <= R[j]) array[k] = L[i++];
        else array[k] = R[j++];
    }
    ////////////////////////////////////
}
```

```
void merge_sort(int* array, int l, int r) {
        if (l < r)
        {
                int q = (l + r) / 2;
                merge_sort(array, l, q);
                merge_sort(array, q + 1, r);
                merge(array, l, q, r);
        }
}
```

Merge Sort

- Pros
  - Nice time complexity O(nlogn)

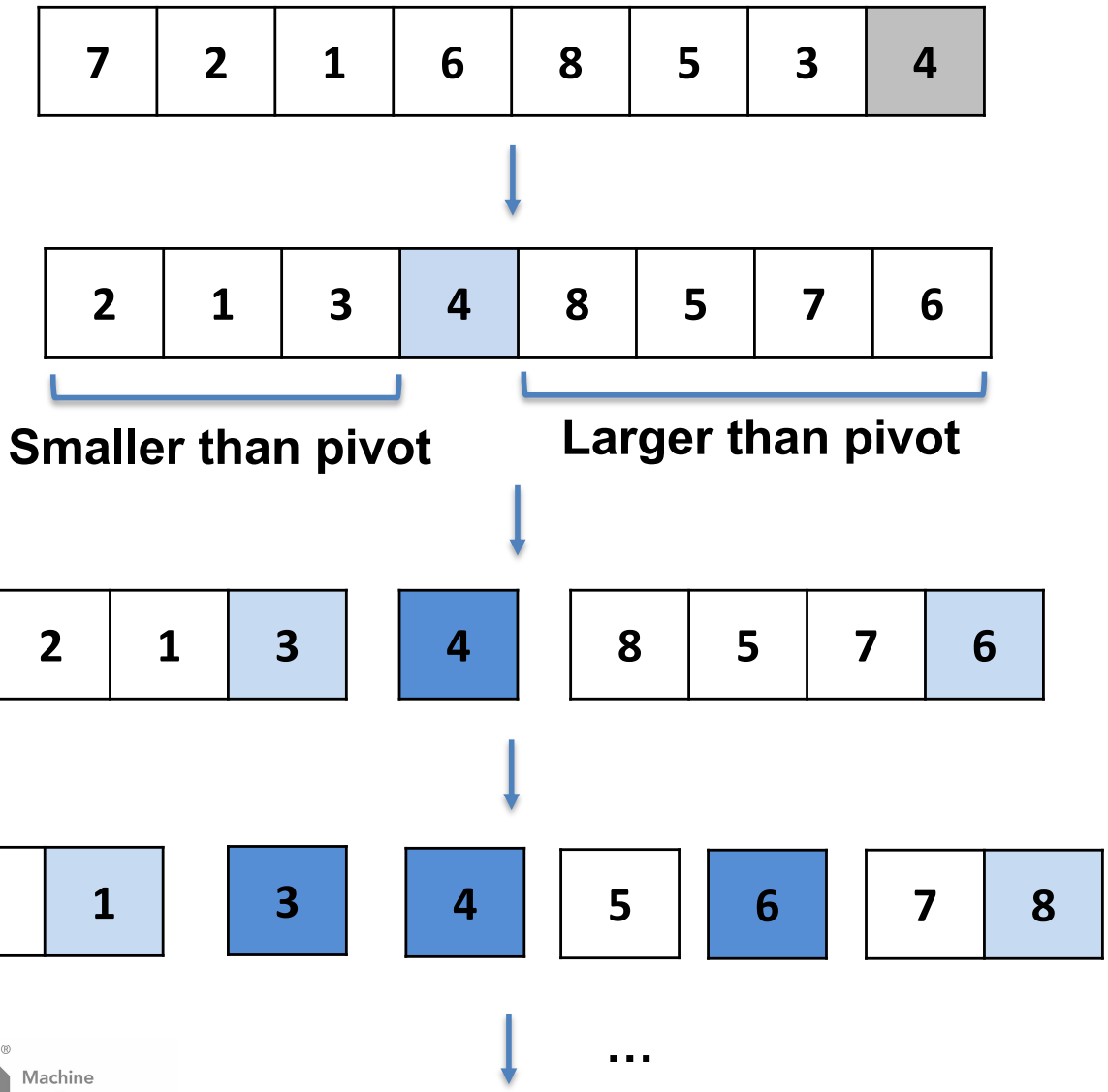


- Cons
  - Additional memory is required to store the divided segments

Outline

- Recursion
  - What is recursion?
- **Incremental Algorithm (Divide and Conquer algorithms)**
  - Merge sort
  - Quick sort
- Linear time sorting algorithm
  - Counting, bucket, radix sorting…

Quick Sort


7 2 1 6 8 5 3 4
Choose a pivot
(end point case)
2 1 3 4 8 5 7 6
Partition
Smaller than pivot
Larger than pivot
2 1 3 4 8 5 7 6
Recursive Sorting on
both left and right of
the pivot
2 1 3 4 5 6 7 8
...

Quick Sort

- **Pseudo Code**
  - ***A*** : given list, ***p*** : start of the list, ***r*** : end of the list, ***q*** : pivot

```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q − 1)
4      QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
1  x = A[r]                          Set end as pivot
2  i = p − 1                         i: For left
3  for j = p to r − 1                j: For right
4      if A[j] ≤ x                   If Right <= Pivot
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1                      Pivot index after
                                     partition
```

Quick Sort

- Why Pseudo Code works?
  - Let's see the **PARTITION(A,p,r)**

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

i: For left
j: For right

Input A
p = 0 , r = 7

7	2	1	6	8	5	3	4

i=-1

x = A[7] = 4
i = p-1 = -1

for- loop (j = 0)
A[j] = 7 > x = 4

Result

7	2	1	6	8	5	3	4

Machine Intelligence & Data science LAB

Quick Sort

- Why Pseudo Code works?
  - Let's see the **PARTITION(A,p,r)**

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Input A

p = 0 , r = 7

7	2	1	6	8	5	3	4

↑ **i=0**

for- loop (j = 1)

A[j] = 2 ≤ x = 4

i = i + 1 = 0

swap (A[i], A[j])

Result

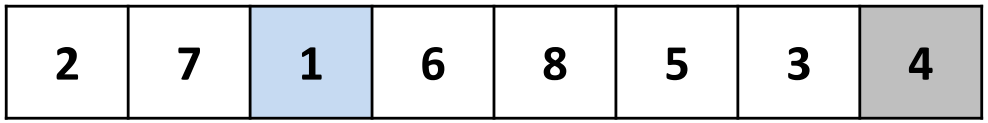
2	7	1	6	8	5	3	4

Quick Sort

- Why Pseudo Code works?
  - Let's see the **PARTITION(A,p,r)**

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

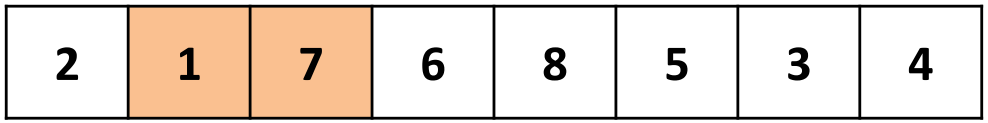
Input A
p = 0 , r = 7



i=1

for- loop (j = 2)
A[j] = 1 ≤ x = 4
i = i + 1 = 1
swap (A[i], A[j])

Result



Quick Sort

- Why Pseudo Code works?
  - Let's see the **PARTITION(A,p,r)**

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Input A

p = 0 , r = 7

2	1	7	6	8	5	3	4

i=1

for- loop (j = 3)

A[j] = 6 > x = 4

Result

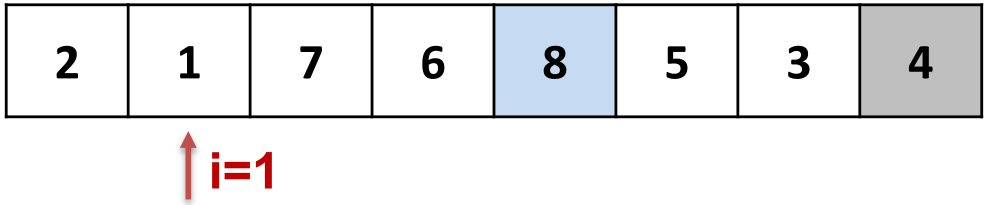
2	1	7	6	8	5	3	4

Quick Sort

- Why Pseudo Code works?
  - Let's see the **PARTITION(A,p,r)**

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

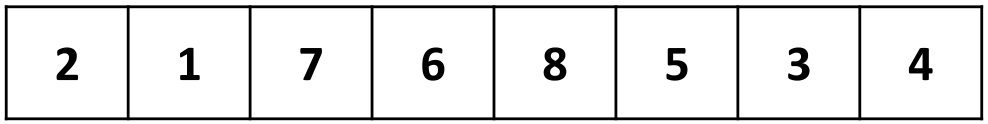
Input A
p = 0 , r = 7



i=1

for- loop (j = 4)
A[j] = 8 > x = 4

Result



Quick Sort

- Why Pseudo Code works?
  - Let's see the **PARTITION(A,p,r)**

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Input A

p = 0 , r = 7

2	1	7	6	8	5	3	4

i=1

for- loop (j = 5)

A[j] = 5 > x = 4

Result

2	1	7	6	8	5	3	4

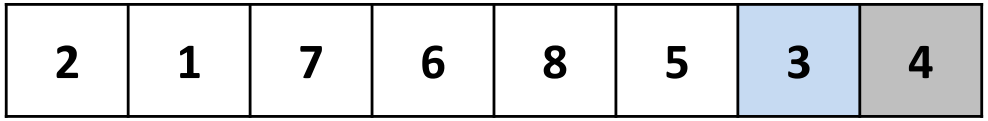
Quick Sort

- Why Pseudo Code works?
  - Let's see the **PARTITION(A,p,r)**

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Input A

p = 0 , r = 7



i=2

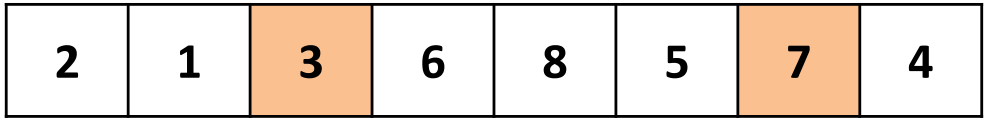
for- loop (j = 6)

A[j] = 3 ≤ x = 4

i = i + 1 = 2

swap (A[i], A[j])

Result

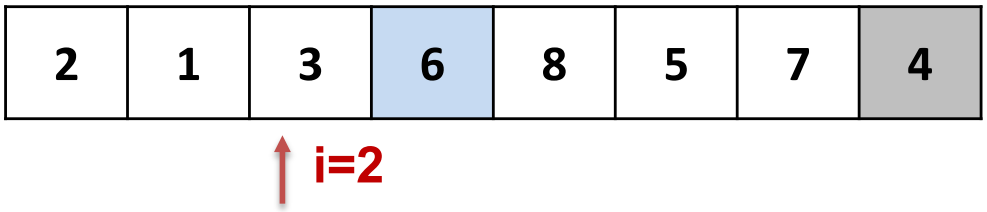


Quick Sort

- Why Pseudo Code works?
  - Let's see the **PARTITION(A,p,r)**

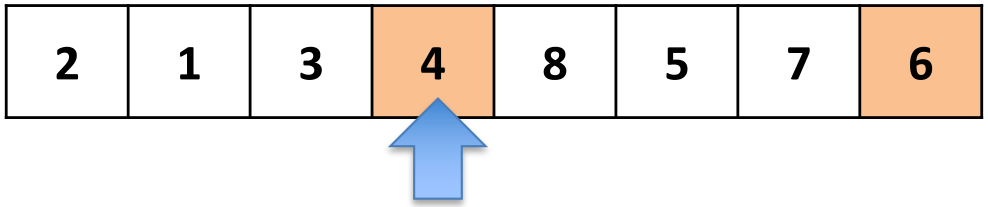
```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

for loop result



swap (A[i+1] = 6, A[r] = 4)

Result

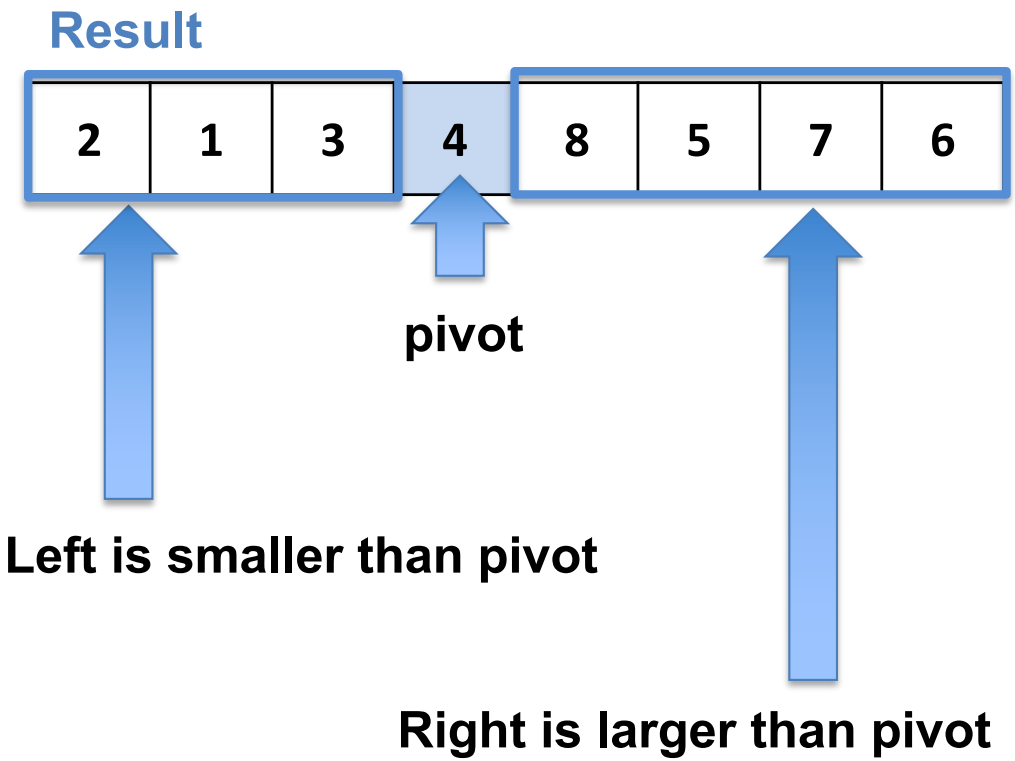


Index i+1 = 3 is pivot

Quick Sort

- Why Pseudo Code works?
  - Let's see the **PARTITION(A,p,r)**

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```



Quick Sort

```
int quick_partition(int* array, int p, int r) {
    int i = p - 1;
    int pv = array[r];

    ////////// IMPLEMENT HERE //////////

    ////////////////////////////////////

    return i+1;
}
```

```
void quick_sort(int* array, int l, int r) {
    int q;
    if (l < r)
    {
        q = quick_partition(array, l, r);
        quick_sort(array, l, q - 1);
        quick_sort(array, q + 1, r);
    }
}
```

Quick Sort

```
int quick_partition(int* array, int p, int r) {
    int i = p - 1;
    int pv = array[r];

    ////////// IMPLEMENT HERE //////////

    for (int j= p ; j < r; j++){
        if (array[j] < pv){
            i++;
            swap (array[i], array[j]);
        }
    }
    swap(array[i+1], array[r]);

    ////////////////////////////////////

    return i+1;
}
```

```
void quick_sort(int* array, int l, int r) {
    int q;
    if (l < r)
    {
        q = quick_partition(array, l, r);
        quick_sort(array, l, q - 1);
        quick_sort(array, q + 1, r);
    }
}
```

Quick Sort

- **Pros**
  - Average time complexity O(nlogn)
  - Best-case time complexity O(nlogn)
  - Do not need additional memory

- **Cons**
  - Not stable
    - Performance depends on pivot selection
    - Worst-case time complexity O(n^2)

Outline

- Recursion
  - What is recursion?
  - Divide and Conquer algorithms:
    - Merge sort
    - Quick sort
- Linear time sorting algorithm
  - **Counting**, bucket, radix sorting…

Counting Sort

- Heap, Merge, Quick Sort: O(nlogn)
- Are these algorithms always optimal?
  - No! (It depends on the problem or resources)

- Linear time sorting algorithm?
  - Counting sort.
  - Radix sort, and so on.

Counting Sort

- Counting sort
  - Phase-1) Given array, find maximum value of the array

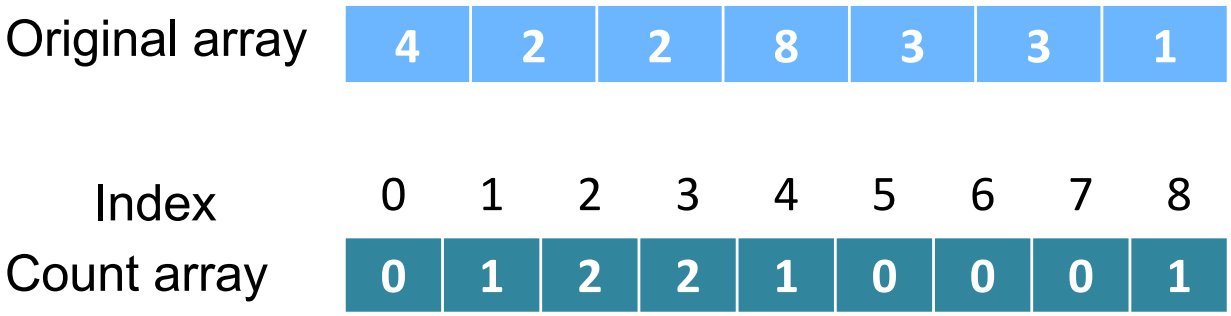
  - Phase-2) Make counting array size of **(maximum value+1)**

    Initialize it with 0.

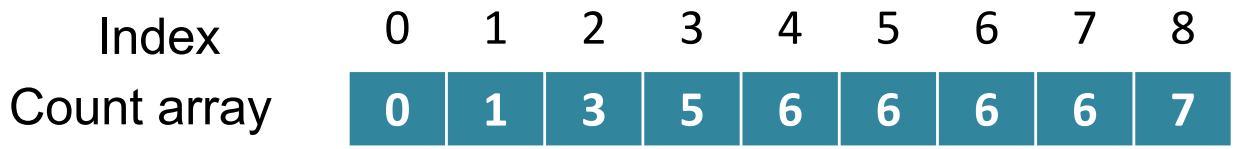
Index	0	1	2	3	4	5	6	7	8
Count array	0	0	0	0	0	0	0	0	0

Counting Sort

- Counting sort
  - Phase-3) Calculate counts of each element and store it in count array.



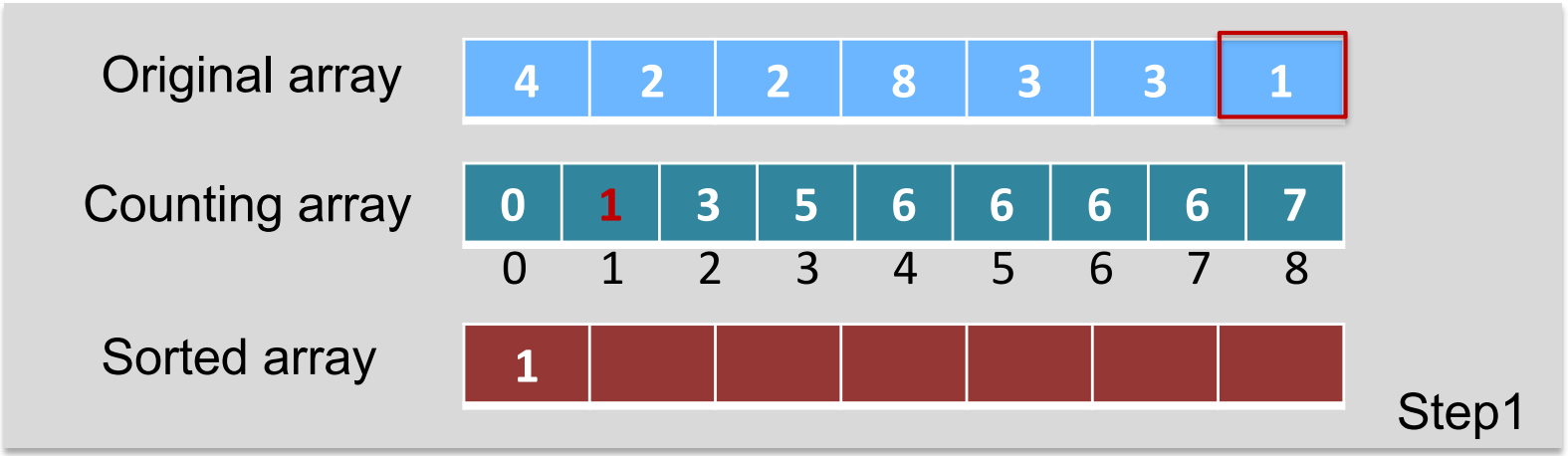
  - Phase-4) Calculate cumulative sum.



From cumulative sum, we can find suitable location for each elements.

Counting Sort

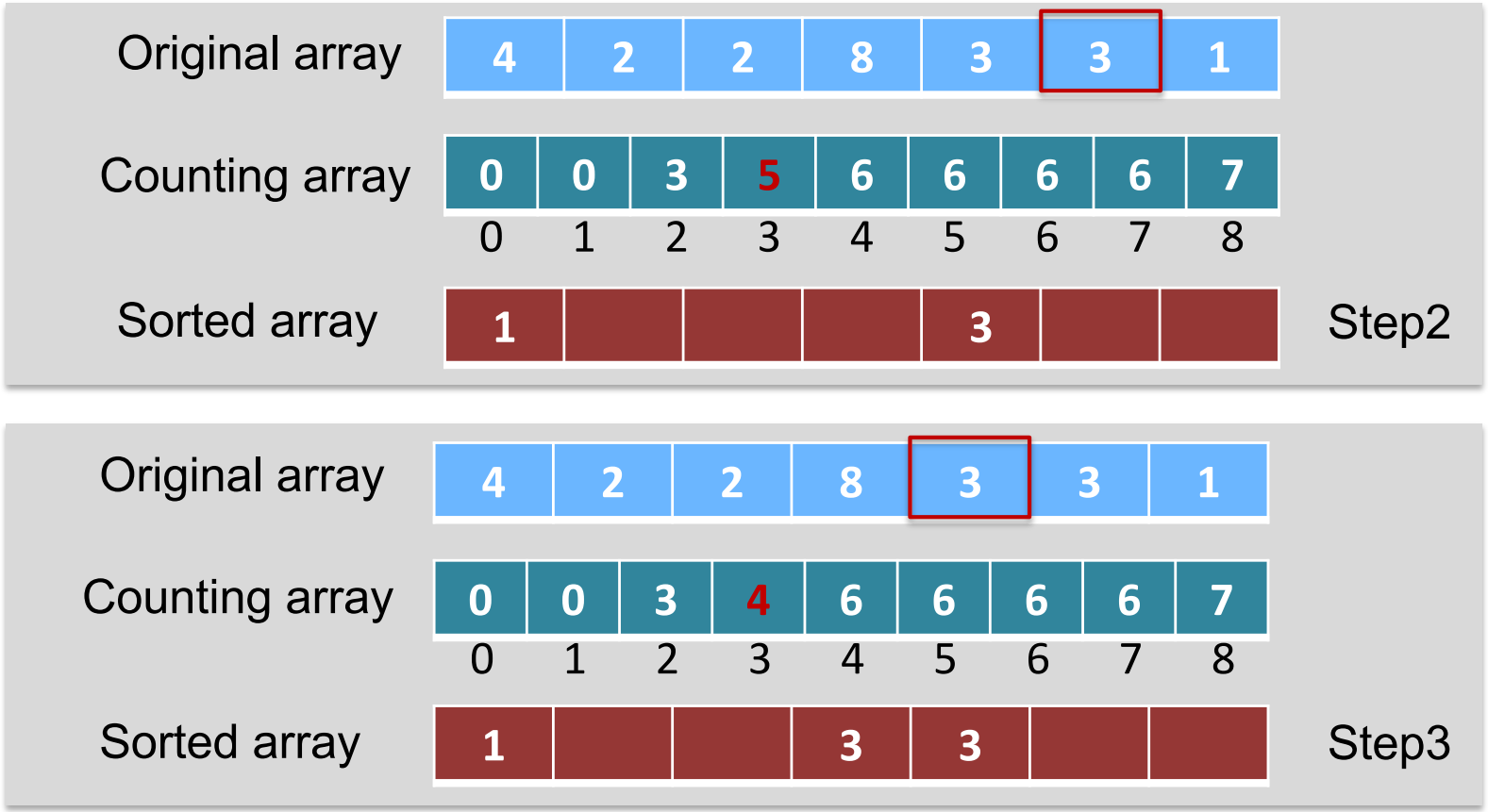
- Counting sort
  - Phase-5) Sorting: Starts with last element of original array.




```
i=size-1
While i>=0:
        sorted[counts[original[i]]-1]= original[i];
        count[original[i]] -= 1;
        i-=1;
```

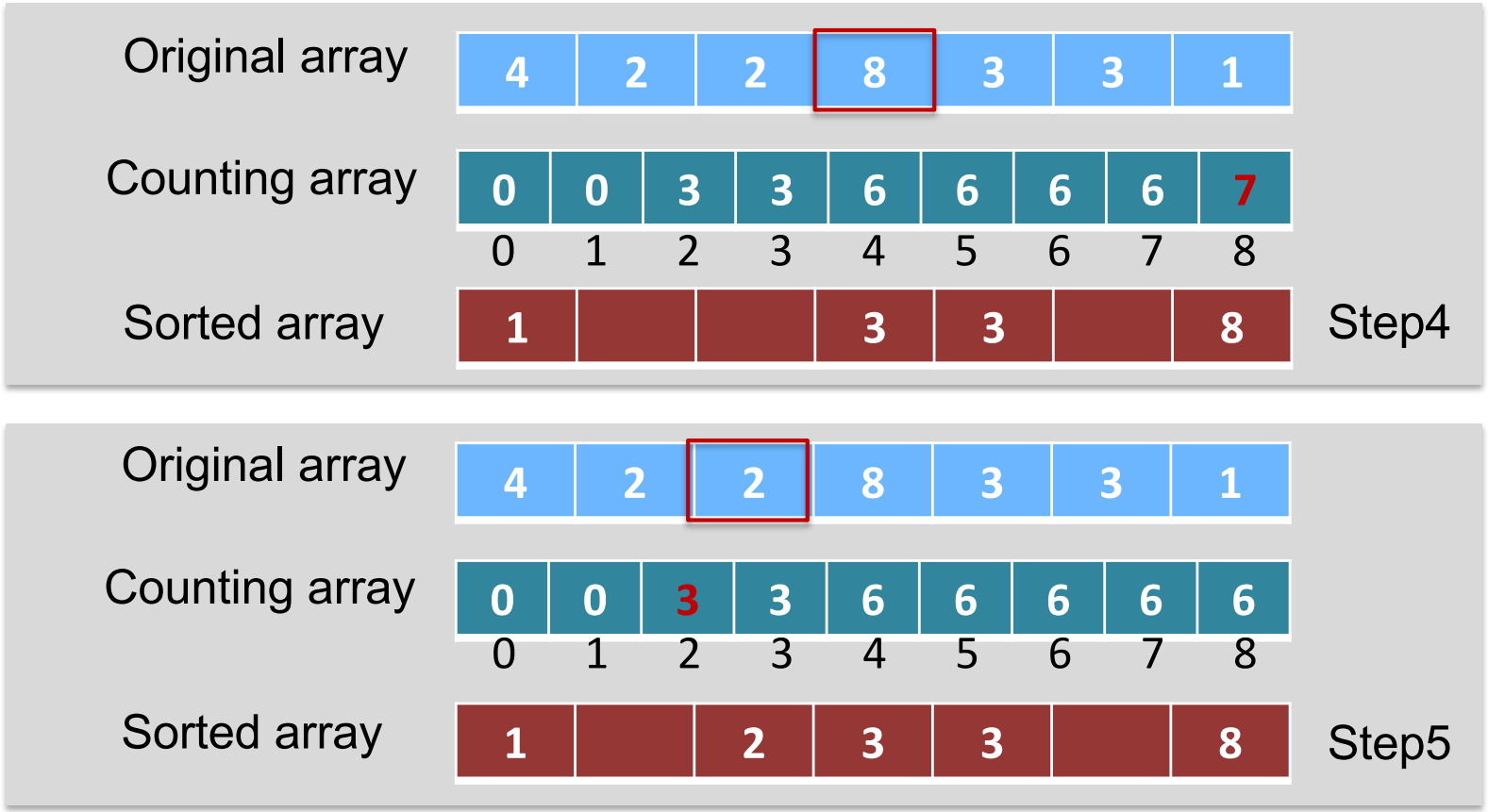
Counting Sort

- Counting sort
  - Phase-5) Sorting:



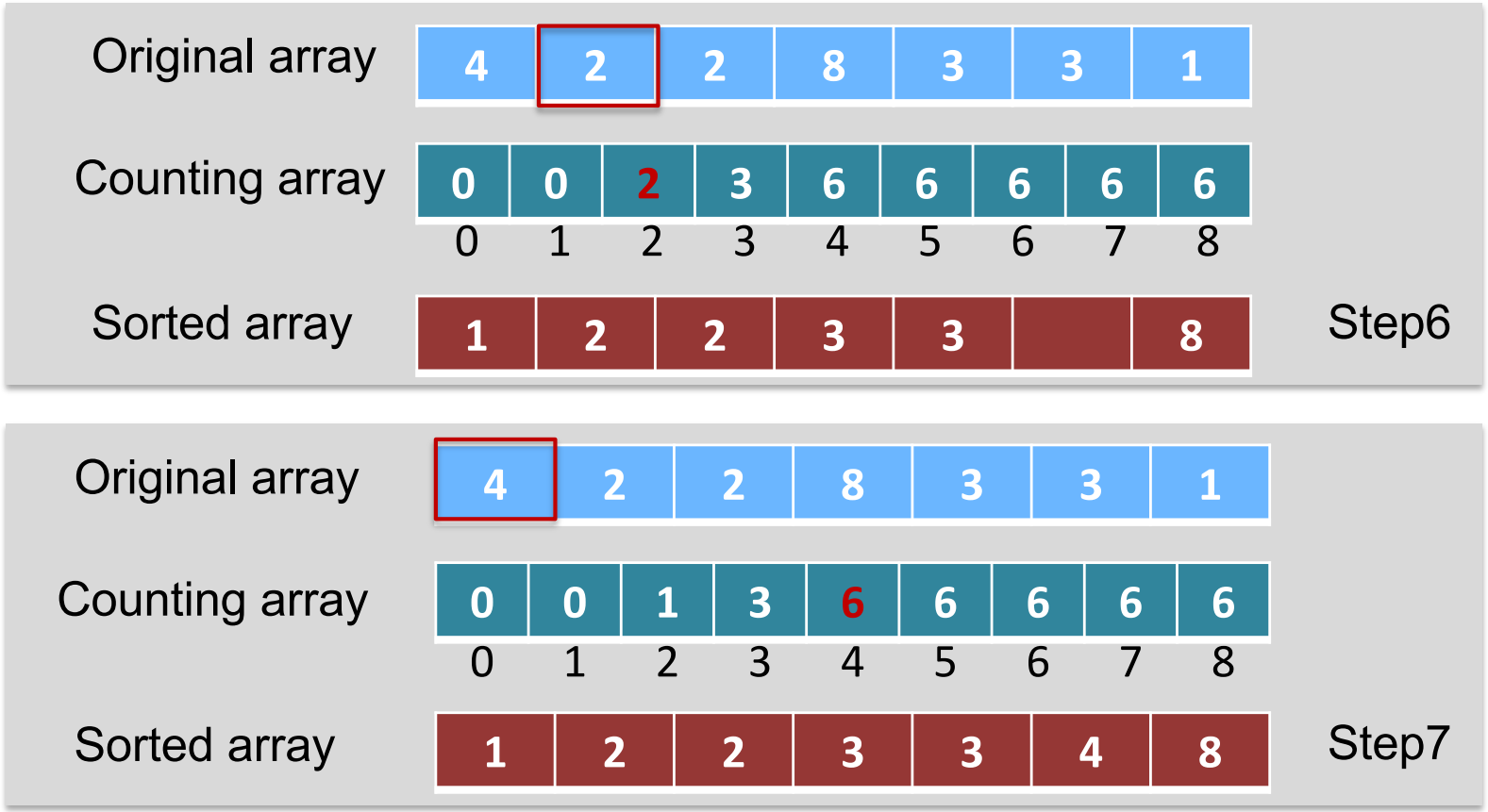
Counting Sort

- Counting sort
  - Phase-5) Sorting:



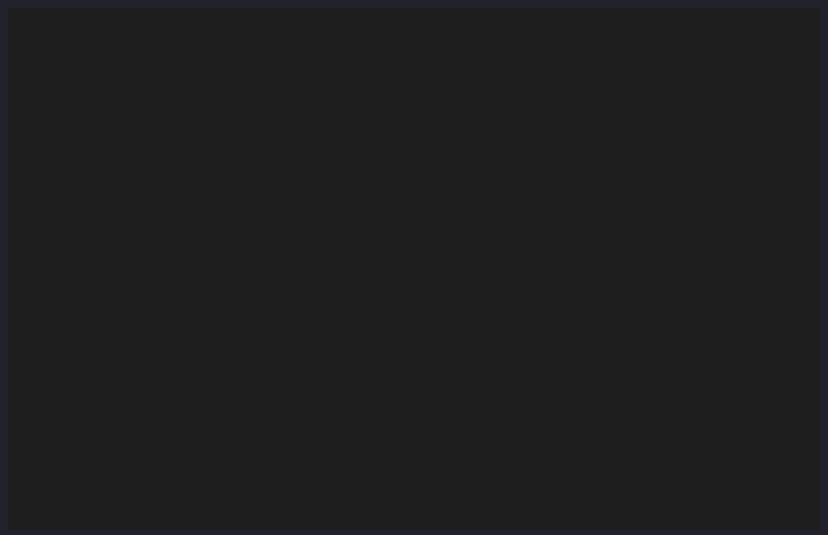
Counting Sort

- Counting sort
  - Phase-5) Sorting:



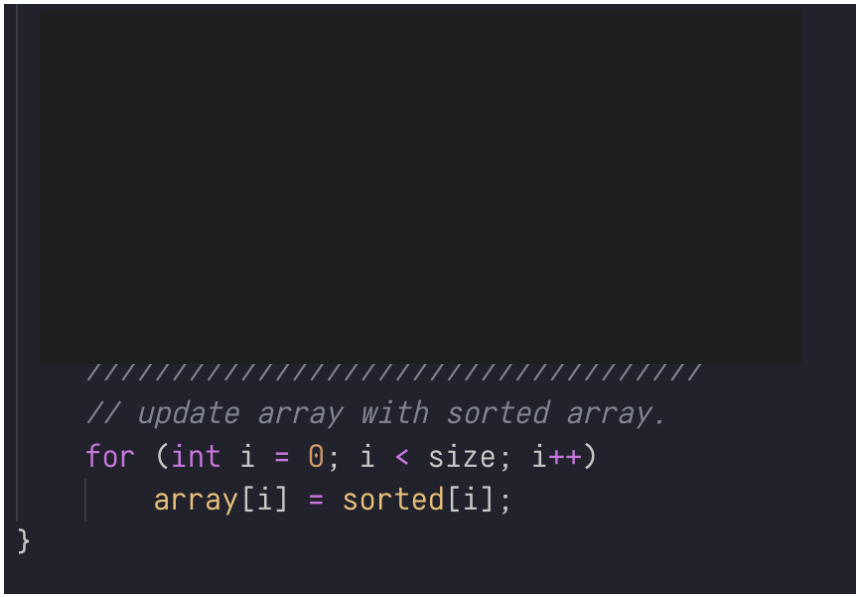
Counting Sort

```
void counting_sort(int* array,  unsigned int size) {
    // Find Maximum value
    int max_value= array [0];
    for (int i = 1; i < size; i++){
        if (array[i] > max_value){
            max_value = array[i];}
    }

    //Initialization
    int counts[max_value+1];
    for (int i=0; i<max_value+1; i++){
        counts[i]= 0;
    }
    int sorted[size];
```



```
    ////////////////////////////////////
```

Implement
Phase 3,4,5 described in previous slides.



```
    ////////////////////////////////////
    // update array with sorted array.
    for (int i = 0; i < size; i++)
        array[i] = sorted[i];
}
```

Counting Sort

```
void counting_sort(int* array,  unsigned int size) {
    // Find Maximum value
    int max_value= array [0];
    for (int i = 1; i < size; i++){
        if (array[i] > max_value){
            max_value = array[i];}
    }

    //Initialization
    int counts[max_value+1];
    for (int i=0; i<max_value+1; i++){
        counts[i]= 0;
    }
    int sorted[size];

    // Counting
    ////////// IMPLEMENT HERE //////////
    for (int i=0; i<size;i++){
        counts[array[i]] += 1;
    }
    ////////////////////////////////////
    // Cumulative sum
    ////////// IMPLEMENT HERE //////////
    for (int i=1; i<=max_value; i++){
        counts[i] += counts[i-1];
    }
    ////////////////////////////////////
```

Implement
Phase 3,4,5 described in previous slides.

```
    // Sorting
    ////////// IMPLEMENT HERE //////////
    int i=size-1;
    while (i>=0) {
        sorted[counts[array[i]]-1] = array[i];
        counts[array[i]] -= 1;
        i -= 1;
    }
    ////////////////////////////////////
    // update array with sorted array.
    for (int i = 0; i < size; i++)
        array[i] = sorted[i];
}
```

Counting Sort

- Pros
  - O($n + k$)
    - where n = array size and k = value range (=max-min)
  - Linear time complexity if k is smaller than the array size n

- Cons
  - But if k is much larger than array size n
    (e.g., n^2, n^3 ... , it is very inefficient)
  - Additional space memory for saving counting array

Assignment

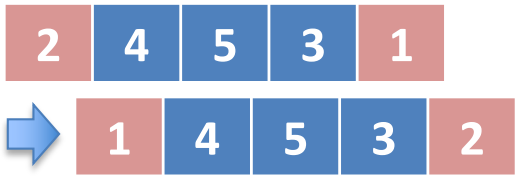
1. Implement **Bubble Sort** and **Insertion** Sort recursively, not iteratively

2. **Stooge Sort** → Next page ..

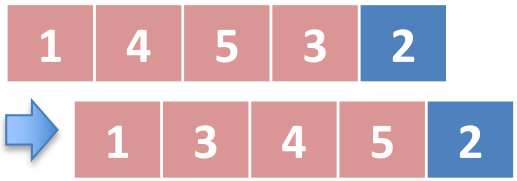
Assignment

2. Implement the stooge-sort algorithm

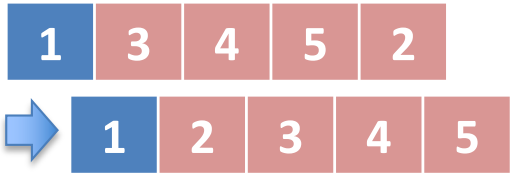
1) Swap initial & last element (since 2 > 1)



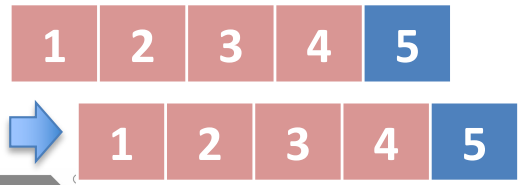
2) Sort the former 2/3 elements



3) Sort the latter 2/3 elements



4) Again, sort the former 2/3 elements



Algorithm:

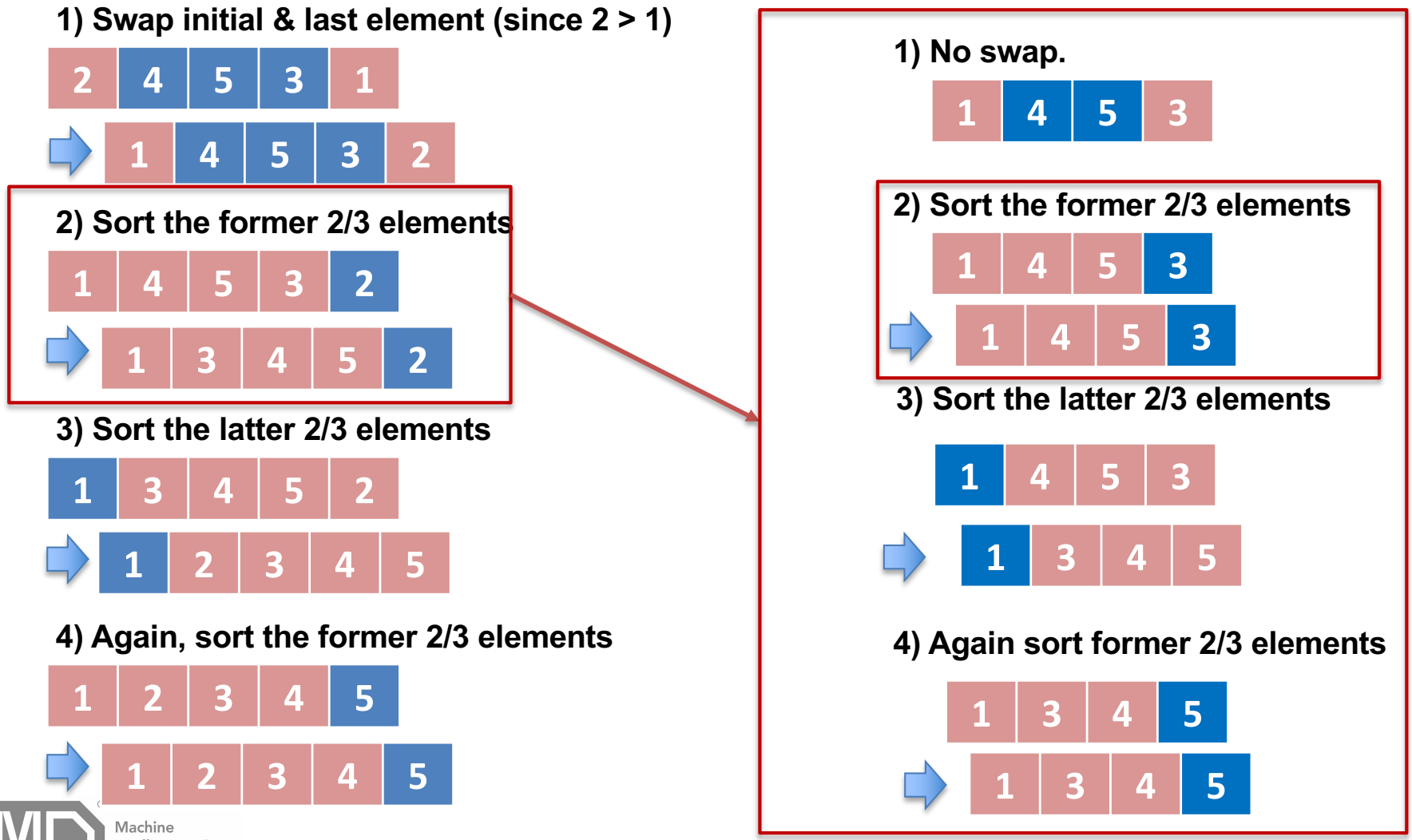
1) If the index 0 value is greater than the last index value, **swap them.**

2) If there are 3 or more elements in the array,
   1. Recursively sort the former 2/3 of the array
   2. Recursively sort the latter 2/3 of the array
   3. Recursively sort the former 2/3 of the array

Machine Intelligence & Data science LAB

Assignment

2. Implement the stooge-sort algorithm



Machine Intelligence & Data science LAB

Assignment

- Submit the implementation codes
  - **No hidden test case** but try it yourself for other test cases
  - Due date : 4/22 14:30 PM
  - Push submit (제출) button on Elice

Attendance Check

- Quiz